CITS3402 - High Performance Computing
Assignment 1: Report

The University of Western Australia

James Oakey (22709404) Brent Tankink (23842541)

Semester 2, 2026

1 Introduction

This document outlines how the puzzling results of the Birthday Paradox can be exploited in order to brute force hash collisions. The goal is to alter the 'nonce' values in two PDF files, each with differing contents, in such a way that the resultant hash is identical. This collision is produced through the combination of nonce variation and the Birthday Attack principle, which proves that collision probability increases rapidly with less trials than is intuitively expected.

2 Birthday-Attack Algorithm

The implemented search strategy uses a Hash-Table data structure to store and search through calculated hashes. Rather than comparing every possible nonce produced from one PDF file to the other, each calculated (hash+file) combination is stored in the hash table. Then when a new hash is generated, it is compared against those previously stored in the table.

The `toy_hash()` algorithm produces 48-bit hashes, giving 2^{48} possible hash outputs. Using the Birthday Paradox, the expected number of trials for a 50% collision probability follows $2^{\sqrt{48}} = 2^{24}$, or 16-17 million values, are required to achieve a 50% collision probability. The probability of no collision occurring can be calculated using the following: $P(\text{No Collision}) \approx e^{-\frac{n^2}{2H}}$, where n is the number of trials required to achieve P probability where there are H possible outputs. For a 99% chance of producing a collision:

$$0.01 = e^{-\frac{n^2}{2\times 2^{48}}} \quad \rightarrow \quad n \approx 51,000,000$$

The loop is limited to 50,000,000 iterations, giving a high probability of producing a collision while limiting the memory being used.

To store and find collisions, the hash table uses linear probing. Each `HashEntry` contains a 64-bit tag and 64-bit nonce, 16 bytes total. The `tag` attribute combines the PDF number with the corresponding 48-bit file hash, allowing the program to distinguish hashes between PDFs. This table is initialised using `calloc`, representing empty slots with 0. The lowest 27 bits of the hash are used to to find the first insertion index to try, where the linear probing strategy is then used if it is occupied.

3 Parallelisation with OpenMP

The serial implementation assings a different nonce to each PDF in every iteration, then calculating the hash for both files. These hashes are then checked against the hash table using `insert_or_match()`. If either hash matches one which was produced by the other PDF, a

collision/match is found and the nonces are returned. If not, then both hashes are inserted and the program continues searching.

| 1. Set and insert nonces into PDF 1 and 2
| 2. Calculate hash of each PDF
| 3. Check hash table
↪ 3a. If match: save nonces and stop
↪ 3b. If not: insert hashes into table and continue

The parallel implementation splits the nonce generation and hashing across multiple threads in a `parallel` region, using a `for` with `schedule(static)`. Static scheduling is the ideal choice for this as each iteration does a similar amount of work.

Each thread creates local copies of the two PDFs so that `set_nonce()` can alter nonces without interference between threads, preventing a data race and threads overwriting each others PDF contents during hash calculations. The hash table is shared between all threads, so the lookup and insertion operations are isolated inside a `critical` region. Without this region, numerous threads could concurrently access the same location in the hash table, potentially leading to the table being inconsistent.

The computationally expensive operations of `set_nonce()` and `toy_hash()` are deliberately left outside the critical region. This minimises the wait time for threads to access the shared hash table and allows most computation to continue simultaneously.

The `found` variable is also shared, and is therefore synchronised in the parallel code using an `atomic read` before each iteration proceeds with their computation (skipping the expensive functions if `found` is true). Inside the critical region it is checked again, as a thread may have found a collision between the initial check and the critical region. This makes sure the first detected collision is used, avoiding unnecessary computation.

4 Memory Usage & Trade-Offs

The hash table size is set as $2^{27} \times 16$ bytes, capping the memory at just over 2GB. This was chosen to ensure that the maximum 50,000,000 trials (for each PDF) can be stored while leaving space to minimise hash table probing conflicts. While dynamic hash table sizing could be more memory efficient, runtime resizing would increase the execution time. Therefore the initial allocation was large to improve speed.

Using a single critical region instead of a partitioned hash table was another trade-off. The region ensures correct access to the shared hash table, but only a single thread can access it at a time. While increasing the thread count can allow greater nonce generation and concurrent hashing, the table access becomes a bottleneck, limiting the speed-up. A partitioned table could reduce this by allowing threads to access partitions simultaneously. This was more complex to implement, so the simpler solution was chosen.

5 Performance Evaluation

5.1 Methodology

To evaluate the performance of our paralell implementation, we timed the the parallel implementation on Kaya. For each pair of PDFs, time was measured for various thread counts (Table 1); certain low-thread counts omitted to avoid exceeding the 15-minute time limit. The search time was measured using `CLOCK_MONOTONIC` around the parallel region, excluding file I/O.

		Threads					
Pair	PDF (KiB)	8	16	32	48	64	96
Search time (s)							
Kilo	64	226.8	195.5	59.7	50.2	63.2	61.2
Mega	128	–	327.7	172.9	61.4	47.3	48.4
Giga	224	–	–	193.7	167.5	42.7	170.6
Tera	336	–	–	191.2	430.3	192.7	132.8
Peta	480	–	–	–	333.9	158.7	111.6
Exa	900	–	–	–	476.3	–	448.8
Estimated nonce pairs checked (millions)							
Kilo	64	14.0	24.2	14.4	18.2	28.8	36.4
Mega	128	–	20.3	21.1	11.0	10.6	13.4
Giga	224	–	–	13.7	17.8	5.9	33.9
Tera	336	–	–	9.0	30.3	18.1	17.2
Peta	480	–	–	–	16.6	9.1	10.6
Exa	900	–	–	–	12.6	–	23.4

Table 1: *Search time and estimated pairs checked per run on Kaya*

5.2 Timings speedup, scalability and bottlenecks

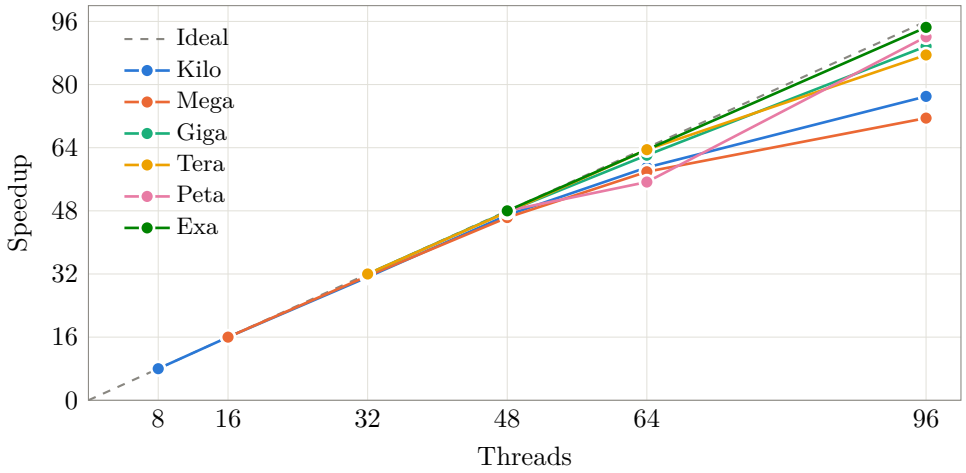


Figure 1: *Speedup from throughput, relative to each pair's lowest measured thread count.*

We observed that raw search times weren't monotonic with thread count, (e.g, Giga took 42.7 s on 64 threads but 170.6 s on 96 threads). We initially suspected this was due to the way NUMA caused latency to the hash table's memory region, but, it turns out that the 96-thread run checked an estimated 33.9M pairs rather than 5.9M, while hashing 1.44× faster. THis suggested that the static scheduling of the colliding nonces is the main factor in the elapsed time, not the

speed of hashing.

Therefore, rather than using elapsed time, speedup was measured as a function of throughput (pairs checked / search time). A serial baseline would have been an inconsiderate use of HPC resources, thus, assuming linear scaling below the lowest measured thread count, speedup was calculated relative to the lowest thread count for each pair. This showed a near-linear speedup up to 48 threads for every pair, with efficiency ranging from 97-100%.

Since a serial baseline would exceed the 15-minute limit, speedup was calculated relative to each pair's lowest measured thread count, assuming linear scaling below it. This revealed near-linear speedup up to 48 threads for every pair, with efficiency ranging from 97-100%.

6 Conclusion